

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 **COURSE NAME:** Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

Industry Problem-Solving Task

Code of Conduct:

I Jeevakrishna. R (192511108) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Overview of Approach

Each of the five scenarios below is drawn from a different security protocol used to protect data in transit or at rest - TLS, PGP, IPSec, S/MIME, and hybrid (AES + RSA) email encryption. For every scenario, the given parameters are first restated, the requested quantities (A-D) are computed step by step in a working table, and the results are then interpreted from an engineering standpoint: what the numbers mean for performance, where the real cost is concentrated, and what a practising network/security engineer would change to improve the outcome. The table below summarises the protocol and the primary metric evaluated in each case before the detailed working is presented.

#	Scenario	Protocol / Mechanism	Primary Metric Evaluated
1	Web server TLS traffic	TLS 1.3 record + handshake	Throughput overhead & handshake bandwidth
2	Enterprise email	PGP (hybrid encryption)	Storage growth & computational load
3	Internal network security	IPSec transport vs. tunnel	Bandwidth overhead & mode selection
4	Organisation-wide identity	S/MIME certificates	Storage & lifecycle-management scale
5	Cloud email service	AES + RSA hybrid encryption	Encryption latency & bottleneck analysis

1. SSL/TLS Overhead and Web Server Performance Optimization

A web server uses TLS 1.3 for secure communication. Each HTTP response carries 2000 bytes of payload. TLS adds 25 bytes (record header + authentication tag) and handshake overhead of 3 KB per session. The server handles 10,000 requests per second, with 1,000 new TLS handshakes per second.

Task:

- Calculate the total data transmitted per second including TLS overhead.
- Determine the percentage overhead due to TLS encryption.
- Estimate bandwidth consumed due to handshake overhead per second.
- Analyze the trade-off between security and latency, and propose optimization techniques (e.g., session resumption, HTTP/2, or QUIC).

Solution

Note on units used throughout this document: 1 KB = 1000 bytes and 1 MB = 1,000,000 bytes (standard networking convention), unless stated otherwise.

Given: payload = 2000 B, TLS record overhead = 25 B/response, handshake overhead = 3 KB (3000 B)/session, request rate = 10,000 req/s, handshake rate = 1,000 sessions/s.

Parameter	Calculation	Result
Data per response (with TLS)	$2000 + 25$	2,025 B
Total response traffic / s	$2,025 \times 10,000$	20,250,000 B/s (20.25 MB/s)
Handshake traffic / s	$3,000 \times 1,000$	3,000,000 B/s (3 MB/s)
A. Total data transmitted / s	$20,250,000 + 3,000,000$	23,250,000 B/s ~ 23.25 MB/s (~186 Mbps)
B. Record-level TLS overhead	$25 / 2000 \times 100$	1.25%
B. Effective overhead (incl. handshake)	$3,250,000 / 20,000,000 \times 100$	16.25%
C. Handshake bandwidth	$1,000 \times 3 \text{ KB}$	3 MB/s (~24 Mbps)

D. Security-latency trade-off: the per-record overhead of TLS 1.3 (1.25%) is small and largely inconsequential, but the handshake cost is not - at 1,000 new sessions every second the server spends 3 MB/s, roughly 13% of total traffic, purely on connection setup rather than useful payload. Every new handshake also adds one to two network round trips of latency before the first byte of data can be sent, which is the real cost users experience. Since encryption itself is cheap, the optimisation problem is really about reducing how often a full handshake has to happen.

- **Session resumption (session tickets / PSK):** reuses a previously negotiated secret so returning clients skip the full handshake, cutting handshake volume and the associated round trip.
- **HTTP/2 multiplexing:** serves many requests over one already-open TLS connection, so the 1,000 handshake/s figure is not driven up further as traffic grows.
- **QUIC / TLS 1.3 0-RTT:** combines the transport and TLS handshake and lets a returning client send data in the very first flight, removing an entire round trip of latency.
- **Connection keep-alive and load-balancer session affinity:** reduce the number of brand-new sessions a single server has to negotiate.

Optimisation Technique	Expected Benefit	Implementation Effort
Session resumption (tickets/PSK)	High - removes full handshake for repeat clients	Low
HTTP/2 multiplexing	Medium - fewer new connections needed	Low
QUIC / TLS 1.3 0-RTT	High - saves a full round trip	Medium

2. PGP-Based Email Encryption Load in Enterprise System

An enterprise email server encrypts emails using PGP. Each email contains a 10 MB attachment. PGP introduces 800 bytes overhead for key encryption and signatures. The system processes 5,000 emails per day.

Task:

- A. Calculate the total data transmitted per day including overhead.
- B. Determine the percentage increase in storage due to PGP.
- C. Analyze the computational overhead during encryption/decryption.
- D. Recommend optimization strategies (e.g., compression, batching, or hardware acceleration).

Solution

Given: attachment = 10 MB/email, PGP overhead = 800 B/email, volume = 5,000 emails/day.

Parameter	Calculation	Result
Data per email (with PGP)	$10,000,000 + 800$	10,000,800 B
A. Total data transmitted / day	$10,000,800 \times 5,000$	50,004,000,000 B ~ 50.0 GB/day
B. Storage increase due to PGP	$800 / 10,000,000 \times 100$	0.008%

C. Computational overhead: although the 800-byte overhead looks negligible in storage terms, it hides two comparatively expensive operations per email - a public-key operation to wrap the one-time symmetric session key (or to sign the message digest), and a hash computation over the full attachment for the digital signature. The bulk 10 MB payload itself is encrypted with a fast symmetric cipher (typically AES under PGP), so its cost scales linearly and stays cheap. The asymmetric step, however, has a comparatively high fixed CPU cost per email; multiplied across 5,000 emails/day this becomes the dominant load on the mail gateway's CPU, not the bulk encryption of attachments.

D. Optimisation strategies:

- **Compress before encrypting:** compressing attachments (where the file type allows it) before PGP encryption reduces both transmitted volume and the data the symmetric cipher has to process.
- **Batch key operations:** where several recipients share a session, wrap the same session key once rather than repeating the asymmetric step per recipient.
- **Hardware acceleration:** offload AES to AES-NI-capable CPUs or a crypto co-processor, and use ECC-based PGP keys in place of RSA to shrink the cost of the public-key step.

Optimisation Technique	Expected Benefit	Implementation Effort
------------------------	------------------	-----------------------

Compress before encrypting	Medium - smaller payload to secure	Low
Batch key operations	Medium - fewer asymmetric ops	Medium
Hardware acceleration (AES-NI / ECC)	High - cuts CPU time per email	Medium

- **Asynchronous / queued processing:** decouple signing and encryption from mail delivery so gateway throughput is not blocked by a burst of large attachments.

3. IPSec Transport vs Tunnel Mode Performance Comparison

A company compares IPSec transport mode and tunnel mode for securing internal communications. Original packet size = 1000 bytes payload + 20-byte header. Transport mode adds 30 bytes, while tunnel mode adds 50 bytes overhead. Traffic volume = 20,000 packets per second.

Task:

- Compute total packet size for both modes.
- Calculate bandwidth usage for each mode per second.
- Compare efficiency and security benefits of both modes.
- Recommend the appropriate mode for internal vs external communication with justification.

Solution

Given: original packet = 1000 B payload + 20 B header = 1,020 B; transport mode adds 30 B; tunnel mode adds 50 B; traffic = 20,000 packets/s.

Metric	Transport Mode	Tunnel Mode
A. Total packet size	$1,020 + 30 = 1,050$ B	$1,020 + 50 = 1,070$ B
B. Bandwidth / s (x 20,000 pkt/s)	21,000,000 B/s ~ 21 MB/s (~168 Mbps)	21,400,000 B/s ~ 21.4 MB/s (~171.2 Mbps)
Overhead relative to original packet	$30 / 1,020 = 2.94\%$	$50 / 1,020 = 4.90\%$

C. Efficiency vs. security: transport mode only encrypts (and authenticates) the payload of the original IP packet, leaving the original IP header exposed. This keeps its overhead low (2.94%) and preserves the original source/destination addressing, which is why it works well between two endpoints that already trust each other. Tunnel mode encapsulates the entire original packet, header included, inside a new IP packet and encrypts the whole thing. The extra encapsulation costs more bandwidth (4.90% overhead, about 400,000 B/s more than transport mode at this traffic volume) but it hides the internal addressing scheme completely from anyone observing the outer packet, which is essential when traffic has to cross an untrusted network such as the public internet.

D. Recommendation:

- **Internal (LAN / host-to-host) traffic:** use transport mode. The network is already trusted and perimeter-controlled, so the lower 2.94% overhead is preferred and there is no need to hide internal IP addressing from itself.
- **External traffic (branch offices, remote access, site-to-site VPN):** use tunnel mode. Encapsulating the full original packet conceals internal network topology and lets private IP address ranges pass safely across the public internet - a security benefit that justifies the extra ~1.96 percentage points of overhead.

Optimisation Technique	Expected Benefit	Implementation Effort
Transport mode for internal traffic	Reduces overhead by ~2 pp on trusted links	Low
Tunnel mode for external/VPN traffic	Full packet confidentiality across untrusted links	Medium

4. S/MIME Certificate Management in Large Organization

An organization manages 12,000 employees using S/MIME certificates. Each certificate is 2 KB, and certificate renewal happens every year. During renewal, both old and new certificates are stored simultaneously for 1 month.

Task:

- Calculate total storage required during the renewal period.
- Estimate certificate distribution overhead across the network.
- Analyze challenges in certificate lifecycle management at scale.
- Propose optimization strategies (e.g., automation, centralized PKI, or short-lived certificates).

Solution

Given: 12,000 employees, certificate size = 2 KB, annual renewal, old and new certificates co-exist for 1 month during renewal.

Parameter	Calculation	Result
Steady-state storage (1 cert/employee)	12,000 x 2 KB	24,000 KB (24 MB)
A. Storage during renewal (old + new)	12,000 x 2 x 2 KB	48,000 KB (48 MB)
B. Distribution overhead (new certs pushed)	12,000 x 2 KB	24,000 KB (24 MB) total; ~800 KB/day if rolled out evenly over 30 days

C. Lifecycle-management challenges at this scale: with 12,000 active identities, certificate management stops being a per-user task and becomes an infrastructure problem in its own right.

- **Revocation growth:** every compromised or replaced certificate adds an entry to the CRL, and OCSP responders must handle a proportionally larger volume of status checks.
- **Renewal synchronisation:** a single fixed annual renewal date creates a load spike; staggering it avoids a rush but complicates tracking which employees are mid-renewal.
- **Storage growth over time:** the temporary doubling seen in part A recurs every renewal cycle and compounds with employee growth.
- **Key backup and recovery:** private keys tied to expired certificates must still be retrievable to decrypt historical email.
- **Human/administrative error:** manual issuance and revocation at this scale is a common source of expired or orphaned certificates and compliance gaps.

D. Optimisation strategies:

- **Automation of issuance and renewal:** auto-enrolment (e.g. via SCEP/ACME-style protocols) removes manual steps and the errors that come with them.
- **Centralised PKI with a self-service portal:** gives IT a single point of policy control, auditing, and revocation across all 12,000 identities.
- **Short-lived certificates:** shorter validity periods (renewed automatically and silently) reduce the exposure window of a compromised key and shrink CRLs.
- **OCSP stapling:** lets the mail server present a signed, cached revocation status, cutting the number of live OCSP queries the client has to make.

Optimisation Technique	Expected Benefit	Implementation Effort
Automated issuance/renewal	High - removes manual error	Medium
Centralised PKI portal	High - single point of control & audit	High
Short-lived certificates	Medium - shrinks exposure window	Medium
OCSP stapling	Medium - reduces live query load	Low

5. Email Encryption Latency in Cloud-Based Systems

A cloud email service encrypts outgoing emails using AES (data) and RSA (key exchange). AES encryption takes 0.2 ms per MB, and RSA key exchange takes 5 ms per email. Each email has 8 MB attachment. The system processes 8,000 emails per hour.

Task:

- Calculate total encryption time per email.
- Estimate total processing time per hour.
- Analyze the performance bottleneck between symmetric and asymmetric operations.
- Recommend optimization strategies (e.g., ECC adoption, parallel processing, or hardware acceleration).

Solution

Given: AES = 0.2 ms/MB, RSA key exchange = 5 ms/email (fixed), attachment = 8 MB, volume = 8,000 emails/hour.

Parameter	Calculation	Result
AES (symmetric) time / email	0.2×8	1.6 ms
A. Total encryption time / email	$1.6 + 5$ (RSA)	6.6 ms
B. Total processing time / hour	$8,000 \times 6.6$ ms	52,800 ms ~ 52.8 s/hour
Share of time - RSA (asymmetric)	$5 / 6.6 \times 100$	75.8%
Share of time - AES (symmetric)	$1.6 / 6.6 \times 100$	24.2%

C. Bottleneck analysis: even though the 8 MB attachment is the far larger amount of data, AES only costs 1.6 ms of it because symmetric encryption throughput scales efficiently with size. RSA key exchange, in contrast, is a fixed-cost operation regardless of attachment size, and here it consumes 75.8% of the total per-email processing time. The bottleneck is therefore the asymmetric key-exchange step, not the bulk data encryption - a pattern typical of hybrid cryptosystems, where the small, fixed-cost public-key operation dominates overall latency once payload sizes are moderate to large.

D. Optimisation strategies:

- **Adopt ECC (e.g. ECDH) instead of RSA for key exchange:** elliptic-curve operations reach equivalent security at a fraction of RSA's computation time, directly cutting the dominant 5 ms cost.
- **Session-key caching / reuse:** where policy allows, reusing a negotiated key for a short window across multiple messages to the same recipient avoids repeating the asymmetric step every time.
- **Parallel processing:** since each email's encryption is independent, distributing the 8,000 emails/hour across multiple cores or worker processes reduces wall-clock processing time proportionally.
- **Hardware acceleration:** dedicated crypto accelerators or HSMs for RSA/ECC operations, alongside AES-NI for the symmetric step, reduce CPU load at scale.

Optimisation Technique	Expected Benefit	Implementation Effort
Adopt ECC/ECDH for key exchange	High - cuts the dominant 5 ms cost	Medium
Session-key caching	Medium - fewer repeated handshakes	Medium
Parallel processing across cores	High - linear reduction in wall-clock time	Low
Hardware acceleration (HSM/AES-NI)	High - offloads CPU-bound crypto	Medium

Overall Conclusion

Across all five scenarios, the underlying pattern is consistent: cryptographic protection itself is rarely the dominant cost - the surrounding overhead (handshakes, key exchange, certificate storage, or asymmetric operations) is what actually limits throughput and latency at scale. The table below consolidates the main overhead driver identified in each problem and the single highest-impact optimisation recommended.

#	Protocol	Main Overhead Driver	Highest-Impact Recommendation
1	TLS 1.3	Handshake setup (not the per-record 1.25% overhead)	Session resumption / TLS 1.3 0-RTT
2	PGP email	Asymmetric key-wrap & signature CPU cost	Hardware-accelerated / ECC-based keys
3	IPSec	Full packet encapsulation in tunnel mode	Use transport mode internally, tunnel mode externally
4	S/MIME PKI	Certificate duplication during renewal overlap	Automated, centralised certificate lifecycle management
5	AES + RSA hybrid	Fixed-cost RSA key exchange (75.8% of latency)	Replace RSA with ECC for key exchange

In every case, the fix is architectural rather than cryptographic: none of the five recommendations involve weakening encryption to gain performance. Instead, each targets the connection-management, key-exchange, or certificate-handling layer that surrounds the cipher itself - session reuse instead of fresh handshakes, elliptic-curve keys instead of RSA, automation instead of manual certificate handling, and mode selection matched to the trust boundary of the network. This is the general principle an engineer should carry from all five exercises: security overhead is manageable once it is measured and attributed to its actual source, rather than assumed to be an unavoidable cost of encryption.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation